My Project

Generated by Doxygen 1.12.0

Chapter 1

Project 4 - Flow Shop Scheduling with NEH and ACO

1.1 Overview

This project benchmarks permutation flow shop scheduling algorithms in C. It implements:

- Standard NEH scheduling
- Blocking-flowshop NEH scheduling
- Ant Colony Optimization (ACO) improvements for both evaluation models
- CSV result generation for benchmark runs
- Doxygen documentation output

The program reads numbered benchmark instances from `input/`, runs the heuristics, and writes results into `Results_471/`.

1.2 Build

On Windows with MinGW:
```
mingw32-make
```

On Linux/macOS:
```
make
```

The Makefile builds:

- `project4.exe` on Windows
- `project4` on Unix systems

1.3 Run

Run the executable with the number of numbered input instances to process:

```
./project4.exe 120
```

or

```
./project4 120
```

This processes `input/1.txt` through `input/120.txt`.

Note: `main.c` currently loads configuration from `config.cfg` in the project root by calling `load_config("config.cfg")`. If that file is missing, the program falls back to built-in defaults.

1.4 Configuration

The optional `config.cfg` file uses simple `key=value` pairs:

```
numfiles=120
ants=30
iterations=300
alpha=1.0
beta=2.0
rho=0.1
```

Default values are defined in `src/config.c` and are used automatically when `config.cfg` is not present.

At the moment, the executable still takes the number of instances to process from the command-line argument. The `numfiles` field is parsed by the config loader, but `main.c` does not currently use it to control the run.

1.5 Input Format

Each benchmark file in `input/` begins with:

```
<num_machines> <num_jobs>
```

The remaining lines contain processing times arranged by machine row. The loader in `src/fileutility.c` converts those rows into per-job processing time arrays used by the schedulers.

1.6 Output

Benchmark results are written to:

- `Results_471/neh.csv`
- `Results_471/aco.csv`
- `Results_471/schedules/schedules.csv`

Object files are written to `build/`.

1.7 Project Structure

1.7.1 Headers

- include/ACO.h
  Public ACO data structures and function declarations.
- include/FLOWSHOP.h
  Core flow shop data structures, standard makespan logic, and NEH interface.
- include/blocking.h
  Blocking-flowshop makespan and blocking NEH declarations.
- include/config.h
  Configuration structure and config loader declaration.
- include/fileutility.h
  Input-file loading interface.
- include/mt19937ar.h
  Mersenne Twister random-number generator interface.
- include/timing.h
  Cross-platform wall-clock timing interface.

1.7.2 Source Files

- src/ACO.c
  ACO implementation, pheromone updates, solution construction, and local search.
- src/blocking.c
  Blocking makespan calculation and blocking NEH heuristic.
- src/config.c
  config.cfg parser with default parameter fallback.
- src/fileutility.c
  Benchmark-instance file loader.
- src/flowshop.c
  Flow shop allocation, standard makespan evaluation, and standard NEH heuristic.
- src/main.c
  Program entry point and benchmark driver.
- src/mt19937ar.c
  MT19937 pseudorandom number generator implementation.
- src/timing.c
  Platform-specific timer implementation.

1.7.3 Other Project Files

- `Makefile`
  Build rules for the executable and output directories.
- `doxyfile`
  Doxygen configuration for generating API documentation.
- `input/`
  Numbered benchmark instances.
- `Results_471/`
  Generated benchmark CSV output.
- `html/`
  Generated HTML Doxygen documentation.
- `latex/`
  Generated LaTeX Doxygen documentation.
- `build/`
  Compiled object files.
- `scripts/`
  Reserved for helper scripts. It is currently empty.

1.8 Documentation

CS471_Proj4.pdf

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 4

Class Documentation

4.1 ACO Struct Reference

Stores algorithm parameters and the pheromone matrix for ACO.

`#include <ACO.h>`

Public Attributes

- int **num_ants**
- int **iterations**
- double **alpha**
- double **beta**
- double **rho**
- double ** **pheromone**

4.1.1 Detailed Description

Stores algorithm parameters and the pheromone matrix for ACO.

The documentation for this struct was generated from the following file:

- include/ACO.h

4.2 Ant Struct Reference

Represents a single ant's candidate solution.

`#include <ACO.h>`

Public Attributes

- int * **perm**
- int **makespan**

4.2.1 Detailed Description

Represents a single ant's candidate solution.

The documentation for this struct was generated from the following file:

- include/ACO.h

4.3 Config Struct Reference

Stores configurable runtime parameters for the solver.

`#include <config.h>`

Public Attributes

- int **numfiles**
- int **ants**
- int **iterations**
- double **alpha**
- double **beta**
- double **rho**

4.3.1 Detailed Description

Stores configurable runtime parameters for the solver.

The documentation for this struct was generated from the following file:

- include/config.h

4.4 Flowshop Struct Reference

Represents a complete flow shop scheduling instance.

`#include <FLOWSHOP.h>`

Public Attributes

- int **num_jobs**
- int **num_machines**
- Job * **jobs**

4.4.1 Detailed Description

Represents a complete flow shop scheduling instance.

The documentation for this struct was generated from the following file:

- include/FLOWSHOP.h

4.5 Job Struct Reference

Represents a single job in a flow shop instance.

`#include <FLOWSHOP.h>`

Public Attributes

- int **job_id**
- int **num_machines**
- int * **processing_times**
- int **makespan**

4.5.1 Detailed Description

Represents a single job in a flow shop instance.

The documentation for this struct was generated from the following file:

- include/FLOWSHOP.h

4.6 JobRank Struct Reference

Stores a job index and its aggregated processing time for ranking.

`#include <FLOWSHOP.h>`

Public Attributes

- int **job_index**
- int **total_time**

4.6.1 Detailed Description

Stores a job index and its aggregated processing time for ranking.

The documentation for this struct was generated from the following file:

- include/FLOWSHOP.h

4.7 Solution Struct Reference

Represents a candidate job ordering and its makespan.

`#include <FLOWSHOP.h>`

Public Attributes

- int * **schedule**
- int **makespan**

4.7.1 Detailed Description

Represents a candidate job ordering and its makespan.

The documentation for this struct was generated from the following file:

- include/FLOWSHOP.h

Chapter 5

File Documentation

5.1 include/ACO.h File Reference

Ant Colony Optimization utilities for flow shop scheduling.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include "FLOWSHOP.h"
#include "blocking.h"
#include "config.h"
```

Classes

- struct Ant

  Represents a single ant's candidate solution.
- struct ACO

  Stores algorithm parameters and the pheromone matrix for ACO.

Typedefs

- typedef int(∗ method) (Flowshop ∗fs, int ∗schedule)

  Evaluation callback used to score a candidate permutation.

Functions

- int evaluate_standard (Flowshop ∗fs, int ∗perm)

  Evaluate a permutation with the standard flow shop makespan model.
- int evaluate_blocking (Flowshop ∗fs, int ∗perm)

  Evaluate a permutation with the blocking flow shop makespan model.
- double ∗∗ aco_create_pheromone (int n, Solution neh)

  Allocate and initialize the pheromone transition matrix.
- void update_pheromone (ACO ∗aco, Ant ∗ants, int num_ants, int n_jobs, int best_makespan)

  Apply evaporation and deposition to the pheromone matrix.
- void aco_free_pheromone (double ∗∗p, int n)

  Free a pheromone matrix created by aco_create_pheromone().
- void construct_solution (ACO ∗aco, Flowshop ∗fs, Ant ∗ant)

  Build a single ant solution from pheromone and heuristic values.
- Solution ant_colony_optimize (Flowshop ∗fs, Solution initial, method choose, Config cfg)

  Run Ant Colony Optimization starting from an initial solution.

5.1.1 Detailed Description

Ant Colony Optimization utilities for flow shop scheduling.

This header declares the core data structures and helper routines used to build, score, and improve ACO solutions for both standard and blocking flow shop variants.

5.1.2 Typedef Documentation

5.1.2.1 method

```
typedef int(* method) (Flowshop *fs, int *schedule)
```

Evaluation callback used to score a candidate permutation.

Parameters

	fs	Pointer to the flow shop instance being solved.
	schedule	Permutation of job indices to evaluate.

Returns

Makespan produced by the supplied permutation.

5.1.3 Function Documentation

5.1.3.1 aco_create_pheromone()

```
double ** aco_create_pheromone (
            int n,
            Solution neh)
```

Allocate and initialize the pheromone transition matrix.

The provided NEH solution is used to seed transitions with additional pheromone before optimization begins.

Parameters

	n	Number of jobs in the problem instance.
	neh	Initial solution used to bias the pheromone matrix.

Returns

Newly allocated pheromone matrix. The caller must free it with aco_free_pheromone().

Allocate and initialize the pheromone transition matrix.

The pheromone matrix is sized (n+1) x n to include a virtual "start" node at index n. The NEH solution is used to seed pheromone on the transitions it uses.

Parameters

	n	Number of jobs.
	neh	Initial NEH solution used to seed pheromone.

Returns

Newly allocated pheromone matrix (caller must free).

5.1.3.2 aco_free_pheromone()

```
void aco_free_pheromone (
            double ** p,
            int n)
```

Free a pheromone matrix created by aco_create_pheromone().

Parameters

	p	Pheromone matrix to release.
	n	Number of jobs used when allocating the matrix.
	p	Pointer to the pheromone matrix.
	n	Number of jobs used for per-row cleanup.

5.1.3.3 ant_colony_optimize()

```
Solution ant_colony_optimize (
            Flowshop * fs,
            Solution initial,
            method choose,
            Config cfg)
```

Run Ant Colony Optimization starting from an initial solution.

Parameters

	fs	Pointer to the flow shop instance.
	initial	Initial solution used to seed the pheromone matrix.
	choose	Evaluation callback for the desired makespan model.
	cfg	Configuration values controlling ACO behavior.

Returns

Best solution found. The returned schedule is dynamically allocated and must be freed by the caller.

Run Ant Colony Optimization starting from an initial solution.

Uses pheromone initialization from the provided initial solution, constructs new ant solutions each iteration, optionally improves them with local search, and updates pheromone based on the best tour found.

Parameters

fs		Flowshop problem instance.
initial		Initial solution used to seed pheromone and as the starting best.
choose		Evaluation method (standard or blocking makespan).
cfg		Configuration values controlling ACO behavior.

Returns

Best solution found by ACO. The caller is responsible for freeing the returned schedule.

5.1.3.4 construct_solution()

```
void construct_solution (
            ACO * aco,
            Flowshop * fs,
            Ant * ant)
```

Build a single ant solution from pheromone and heuristic values.

The ant's permutation storage must already be allocated for the number of jobs in the flow shop instance.

Parameters

aco		Pointer to the ACO state containing the pheromone matrix.
fs		Pointer to the flow shop instance.
ant		Ant object that receives the constructed permutation.

Build a single ant solution from pheromone and heuristic values.

This uses a job-to-job transition graph and selects the next job by roulette wheel selection using pheromone^alpha * heuristic^beta.

Parameters

aco		Pointer to the ACO algorithm state.
fs		Flowshop problem instance.
ant		Ant object to fill with a constructed permutation.

5.1.3.5 evaluate_blocking()

```
int evaluate_blocking (
            Flowshop * fs,
            int * perm)
```

Evaluate a permutation with the blocking flow shop makespan model.

Parameters

fs		Pointer to the flow shop instance.
perm		Permutation of job indices to evaluate.

Returns

Blocking makespan for the permutation.

Evaluate a permutation with the blocking flow shop makespan model.

Parameters

	fs	Pointer to the Flowshop instance.
	perm	Permutation of job indices representing the schedule.

Returns

The makespan for the given schedule (blocking variant).

5.1.3.6 evaluate_standard()

```
int evaluate_standard (
            Flowshop * fs,
            int * perm)
```

Evaluate a permutation with the standard flow shop makespan model.

Parameters

	fs	Pointer to the flow shop instance.
	perm	Permutation of job indices to evaluate.

Returns

Standard makespan for the permutation.

Evaluate a permutation with the standard flow shop makespan model.

Parameters

	fs	Pointer to the Flowshop instance.
	perm	Permutation of job indices representing the schedule.

Returns

The makespan for the given schedule.

5.1.3.7 update_pheromone()

```
void update_pheromone (
            ACO * aco,
            Ant * ants,
            int num_ants,
            int n_jobs,
            int best_makespan)
```

Apply evaporation and deposition to the pheromone matrix.

Parameters

aco	Pointer to the ACO state to update.
ants	Array of ants whose solutions contribute deposited pheromone.
num_ants	Number of ants in `ants`.
n_jobs	Number of jobs in the instance.
best_makespan	Best makespan used to scale pheromone deposition.

Apply evaporation and deposition to the pheromone matrix.

Deposition is scaled by (best_makespan / ant_makespan) so better solutions deposit more pheromone.

Parameters

aco	Pointer to the ACO state containing pheromone and rho.
ants	Array of ants whose tours are used for deposition.
num_ants	Number of ants in the array.
n_jobs	Number of jobs defining pheromone matrix dimensions.
best_makespan	Best makespan found in this iteration for scaling.

5.2 ACO.h

Go to the documentation of this file.

```
00001 #ifndef ACO_H
00002 #define ACO_H
00003
00004 #include <stdio.h>
00005 #include <stdlib.h>
00006 #include <string.h>
00007 #include <math.h>
00008 #include "FLOWSHOP.h"
00009 #include "blocking.h"
00010 #include "config.h"
00011
00028 typedef int (*method)(
00029     Flowshop *fs,
00030     int *schedule
00031 );
00032
00036 typedef struct {
00037     int *perm;
00038     int makespan;
00039 } Ant;
00040
00044 typedef struct {
00045     int num_ants;
00046     int iterations;
00047
00048     double alpha;
00049     double beta;
00050     double rho;
00051
00052     double **pheromone;
00053 } ACO;
00054
00062 int evaluate_standard(Flowshop *fs, int *perm);
00063
00071 int evaluate_blocking(Flowshop *fs, int *perm);
00072
00084 double **aco_create_pheromone(int n, Solution neh);
00085
00095 void update_pheromone(ACO *aco, Ant *ants, int num_ants, int n_jobs, int best_makespan);
00096
00103 void aco_free_pheromone(double **p, int n);
00104
00115 void construct_solution(ACO *aco, Flowshop *fs, Ant *ant);
00116
00127 Solution ant_colony_optimize(Flowshop *fs, Solution initial, method choose, Config cfg);
00128
00129 #endif /* ACO_H */
```

5.3 include/blocking.h File Reference

Blocking flow shop scheduling interface.

```
#include <string.h>
#include "FLOWSHOP.h"
```

Functions

- int compare_neh (const void *a, const void *b)

  Compare two JobRank values by descending total processing time.
- int flowshop_calculate_makespan_blocking (Flowshop *fs, int *schedule, int count)

  Compute the blocking flow shop makespan for a partial schedule.
- Solution flowshop_schedule_blocking (Flowshop *fs)

  Build a blocking NEH schedule for the given flow shop instance.

5.3.1 Detailed Description

Blocking flow shop scheduling interface.

This header declares the blocking makespan calculation and the blocking variant of the NEH scheduling heuristic.

5.3.2 Function Documentation

5.3.2.1 compare_neh()

```
int compare_neh (
            const void * a,
            const void * b)
```

Compare two JobRank values by descending total processing time.

This comparator is shared with the blocking NEH routine when ranking jobs.

Parameters

	a	Pointer to the first JobRank.
	b	Pointer to the second JobRank.

Returns

Negative, zero, or positive according to qsort comparator rules.

Compare two JobRank values by descending total processing time.

Parameters

	a	Pointer to the first JobRank.
	b	Pointer to the second JobRank.

Returns

Negative, zero, or positive according to qsort comparator rules.

5.3.2.2 flowshop_calculate_makespan_blocking()

```
int flowshop_calculate_makespan_blocking (
            Flowshop * fs,
            int * schedule,
            int count)
```

Compute the blocking flow shop makespan for a partial schedule.

Parameters

	fs	Flow shop instance being evaluated.
	schedule	Permutation of job indices.
	count	Number of leading jobs from `schedule` to evaluate.

Returns

Blocking makespan of the partial schedule.

5.3.2.3 flowshop_schedule_blocking()

```
Solution flowshop_schedule_blocking (
            Flowshop * fs)
```

Build a blocking NEH schedule for the given flow shop instance.

Parameters

	fs	Flow shop instance to schedule.

Returns

Best blocking solution produced by the heuristic. The returned schedule is dynamically allocated and must be freed by the caller.

Build a blocking NEH schedule for the given flow shop instance.

Jobs are ranked by total processing time and inserted iteratively at the position that minimizes the blocking makespan, with random tie-breaking.

Parameters

	fs	Flow shop instance to schedule.

Returns

Solution produced by the blocking NEH heuristic.

5.4 blocking.h

Go to the documentation of this file.

```
00001
00009 #include <string.h>
00010 #include "FLOWSHOP.h"
00011
00021 int compare_neh(const void *a, const void *b);
00022
00031 int flowshop_calculate_makespan_blocking(Flowshop *fs, int *schedule, int count);
00032
00040 Solution flowshop_schedule_blocking(Flowshop *fs);
```

5.5 include/config.h File Reference

Configuration data and loader interface for Project 4.

Classes

- struct Config

  Stores configurable runtime parameters for the solver.

Functions

- Config load_config (const char *filename)

  Load configuration values from a text file.

5.5.1 Detailed Description

Configuration data and loader interface for Project 4.

This header defines the runtime parameters used by the benchmark driver and the Ant Colony Optimization routine, along with the function that loads those parameters from a configuration file.

5.5.2 Function Documentation

5.5.2.1 load_config()

```
Config load_config (
            const char * filename)
```

Load configuration values from a text file.

The configuration file is expected to contain one numeric `key=value` pair per line. If the file cannot be opened, built-in default values are returned instead.

Parameters

	filename	Path to the configuration file.

Returns

Loaded configuration values, or defaults if loading fails.

Supported keys include `numfiles`, `ants`, `iterations`, `alpha`, `beta`, and `rho`. Missing files cause the function to return a Config structure populated with default values.

Parameters

	filename	Path to the configuration file to read.

Returns

Parsed configuration values, or defaults if the file cannot be opened.

5.6 config.h

Go to the documentation of this file.

```
00001 #ifndef CONFIG_H
00002 #define CONFIG_H
00003
00016 typedef struct {
00017     int numfiles;
00018     int ants;
00019     int iterations;
00020     double alpha;
00021     double beta;
00022     double rho;
00023 } Config;
00024
00035 Config load_config(const char *filename);
00036
00037 #endif /* CONFIG_H */
```

5.7 include/fileutility.h File Reference

Input helpers for loading flow shop instances from disk.

```
#include <stdio.h>
#include <stdlib.h>
#include "FLOWSHOP.h"
```

Functions

- Flowshop * loadVector (const char *filename)

  Load a flow shop instance from a text file.

5.7.1 Detailed Description

Input helpers for loading flow shop instances from disk.

5.7.2 Function Documentation

5.7.2.1 loadVector()

```
Flowshop * loadVector (
            const char * filename)
```

Load a flow shop instance from a text file.

The expected format begins with the machine and job counts, followed by processing times arranged by machine row.

Parameters

	filename	Path to the input file.

Returns

Newly allocated flow shop instance, or NULL if the file cannot be opened or parsed.

Load a flow shop instance from a text file.

The file format begins with the number of machines and jobs, followed by a matrix of processing times stored one machine row at a time.

Parameters

	filename	Path to the input file to read.

Returns

Newly allocated flow shop instance, or NULL if loading fails.

5.8 fileutility.h

Go to the documentation of this file.

```
00001
00006 #include <stdio.h>
00007 #include <stdlib.h>
00008 #include "FLOWSHOP.h"
00009
00020 Flowshop* loadVector(const char *filename);
```

5.9 include/FLOWSHOP.h File Reference

Core data structures and NEH-based routines for flow shop scheduling.

```
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
```

Classes

- struct Job

  Represents a single job in a flow shop instance.
- struct Solution

  Represents a candidate job ordering and its makespan.
- struct Flowshop

  Represents a complete flow shop scheduling instance.
- struct JobRank

  Stores a job index and its aggregated processing time for ranking.

Functions

- Flowshop * flowshop_create (int num_machines, int num_jobs)

  Allocate and initialize a flow shop instance.
- void flowshop_free (Flowshop *fs)

  Release all memory owned by a flow shop instance.
- void flowshop_add_job (Flowshop *fs, int job_id, int *times)

  Add a job and its processing times to a flow shop instance.
- int compare_jobs_desc (const void *a, const void *b)

  Compare two entries for descending-order job sorting.
- int compare_neh (const void *a, const void *b)

  Compare two JobRank values by descending total processing time.
- int flowshop_calculate_makespan_partial (Flowshop *fs, int *schedule, int n_jobs)

  Compute the standard flow shop makespan for a partial schedule.
- Solution flowshop_schedule (Flowshop *fs)

  Build a standard NEH schedule for the given flow shop instance.

5.9.1 Detailed Description

Core data structures and NEH-based routines for flow shop scheduling.

This header defines the job, solution, and problem-instance structures used throughout the project, along with the standard flow shop scheduling helpers and makespan calculations.

5.9.2 Function Documentation

5.9.2.1 compare_jobs_desc()

```
int compare_jobs_desc (
            const void * a,
            const void * b)
```

Compare two entries for descending-order job sorting.

Parameters

	a	Pointer to the first element.
	b	Pointer to the second element.

Returns

Negative, zero, or positive according to qsort comparator rules.

5.9.2.2 compare_neh()

```
int compare_neh (
            const void * a,
            const void * b)
```

Compare two JobRank values by descending total processing time.

Parameters

	a	Pointer to the first JobRank.
	b	Pointer to the second JobRank.

Returns

Negative, zero, or positive according to qsort comparator rules.

Compare two JobRank values by descending total processing time.

Parameters

	a	Pointer to the first JobRank.
	b	Pointer to the second JobRank.

Returns

Negative, zero, or positive according to qsort comparator rules.

5.9.2.3 flowshop_add_job()

```
void flowshop_add_job (
            Flowshop * fs,
            int job_id,
            int * times)
```

Add a job and its processing times to a flow shop instance.

Parameters

	fs	Flow shop instance receiving the new job.
	job_id	External identifier assigned to the job.
	times	Array of processing times, one per machine.

Add a job and its processing times to a flow shop instance.

Parameters

	fs	Flow shop instance receiving the job.
	job_id	External identifier assigned to the job.
	times	Processing-time array with one entry per machine.

5.9.2.4 flowshop_calculate_makespan_partial()

```
int flowshop_calculate_makespan_partial (
            Flowshop * fs,
            int * schedule,
            int n_jobs)
```

Compute the standard flow shop makespan for a partial schedule.

Parameters

	fs	Flow shop instance being evaluated.
	schedule	Permutation of job indices.
	n_jobs	Number of leading jobs from `schedule` to evaluate.

Returns

Makespan of the partial schedule.

Compute the standard flow shop makespan for a partial schedule.

Parameters

	fs	Flow shop instance being evaluated.
	schedule	Permutation of job indices.
	n_jobs	Number of leading jobs from `schedule` to evaluate.

Returns

Makespan of the partial schedule.

5.9.2.5 flowshop_create()

```
Flowshop * flowshop_create (
            int num_machines,
            int num_jobs)
```

Allocate and initialize a flow shop instance.

Parameters

	num_machines	Number of machines in the instance.
	num_jobs	Number of jobs in the instance.

Returns

Newly allocated flow shop instance, or NULL if allocation fails.

Parameters

	num_machines	Number of machines in the instance.
	num_jobs	Number of jobs in the instance.

Returns

Newly allocated flow shop instance.

5.9.2.6 flowshop_free()

```
void flowshop_free (
            Flowshop * fs)
```

Release all memory owned by a flow shop instance.

Parameters

	fs	Flow shop instance to free.

5.9.2.7 flowshop_schedule()

```
Solution flowshop_schedule (
            Flowshop * fs)
```

Build a standard NEH schedule for the given flow shop instance.

Parameters

	fs	Flow shop instance to schedule.

Returns

Best solution produced by the NEH heuristic. The returned schedule is dynamically allocated and must be freed by the caller.

Build a standard NEH schedule for the given flow shop instance.

Jobs are ranked by total processing time and inserted iteratively at the position that minimizes the partial makespan, with MT19937 tie-breaking.

Parameters

	fs	Flow shop instance to schedule.

Returns

Solution produced by the standard NEH heuristic.

5.10 FLOWSHOP.h

Go to the documentation of this file.

```
00001 #ifndef FLOWSHOP_H
00002 #define FLOWSHOP_H
00003
00004 #include <stdint.h>
00005 #include <stdlib.h>
00006 #include <string.h>
00007
00020 typedef struct {
00021     int job_id;
00022     int num_machines;
00023     int *processing_times;
00024     int makespan;
00025 } Job;
00026
00030 typedef struct {
00031     int *schedule;
00032     int makespan;
00033 } Solution;
00034
00038 typedef struct {
00039     int num_jobs;
00040     int num_machines;
```

```
00041     Job *jobs;
00042 } Flowshop;
00043
00047 typedef struct {
00048     int job_index;
00049     int total_time;
00050 } JobRank;
00051
00059 Flowshop* flowshop_create(int num_machines, int num_jobs);
00060
00066 void flowshop_free(Flowshop *fs);
00067
00075 void flowshop_add_job(Flowshop *fs, int job_id, int *times);
00076
00084 int compare_jobs_desc(const void *a, const void *b);
00085
00093 int compare_neh(const void *a, const void *b);
00094
00103 int flowshop_calculate_makespan_partial(Flowshop *fs, int *schedule, int n_jobs);
00104
00112 Solution flowshop_schedule(Flowshop *fs);
00113
00114
00115 #endif /* FLOWSHOP_H */
```

5.11 include/mt19937ar.h File Reference

Mersenne Twister MT19937 random number generator interface.

`#include <stdint.h>`

Functions

- void init_genrand (uint32_t s)

  Initializes the random number generator with a seed.
- uint32_t genrand_int32 (void)

  Generates a 32-bit unsigned random integer.
- double genrand_real2 (void)

  Generates a floating-point random number in the range [0, 1).

5.11.1 Detailed Description

Mersenne Twister MT19937 random number generator interface.

This header declares functions for initializing and using the MT19937 pseudorandom number generator.

5.11.2 Function Documentation

5.11.2.1 genrand_int32()

```
uint32_t genrand_int32 (
            void )
```

Generates a 32-bit unsigned random integer.

Returns

Random 32-bit unsigned integer.

If the generator state has been exhausted, the state is regenerated automatically.

Returns

Random 32-bit unsigned integer.

5.11.2.2 genrand_real2()

```
double genrand_real2 (
            void )
```

Generates a floating-point random number in the range [0, 1).

Returns

Random double in the range [0,1).

Generates a floating-point random number in the range [0, 1).

Uses a 32-bit integer random value scaled to the unit interval.

Returns

Random double in the range [0,1).

5.11.2.3 init_genrand()

```
void init_genrand (
            uint32_t s)
```

Initializes the random number generator with a seed.

Parameters

	s	Seed value.

Initializes the random number generator with a seed.

This function must be called before using the random number generator unless a default seed is acceptable.

Parameters

	s	Seed value.

5.12 mt19937ar.h

Go to the documentation of this file.

```
00001 #ifndef MT19937AR_H
00002 #define MT19937AR_H
00003
00004 #include <stdint.h>
00005
00019 void init_genrand(uint32_t s);
00020
00026 uint32_t genrand_int32(void);
00027
00033 double genrand_real2(void);
00034
00035 #endif /* MT19937AR_H */
```

5.13 include/timing.h File Reference

Wall-clock timing utility interface.

Functions

- double now_ms (void)

  Returns the current wall-clock time in milliseconds.

5.13.1 Detailed Description

Wall-clock timing utility interface.

This header declares a portable function for obtaining the current wall-clock time in milliseconds.

5.13.2 Function Documentation

5.13.2.1 now_ms()

```
double now_ms (
            void )
```

Returns the current wall-clock time in milliseconds.

The underlying implementation is platform-specific and uses high-resolution timers when available.

Returns

Current time in milliseconds.

Returns the current wall-clock time in milliseconds.

Uses CLOCK_MONOTONIC when available, otherwise falls back to clock().

Returns

Current time in milliseconds.

5.14 timing.h

Go to the documentation of this file.

```
00001 #ifndef TIMING_H
00002 #define TIMING_H
00003
00020 double now_ms(void);
00021
00022 #endif /* TIMING_H */
```

5.15 src/ACO.c File Reference

Ant Colony Optimization implementation for flow shop scheduling.

```
#include "ACO.h"
#include "FLOWSHOP.h"
#include "mt19937ar.h"
#include <math.h>
#include <string.h>
#include <stdlib.h>
#include "config.h"
```

Functions

- int evaluate_standard (Flowshop *fs, int *perm)

  Evaluate a schedule using the standard (non-blocking) makespan calculation.
- int evaluate_blocking (Flowshop *fs, int *perm)

  Evaluate a schedule using the blocking makespan calculation.
- double ** aco_create_pheromone (int n, Solution neh)

  Create and initialize the pheromone matrix for the Ant Colony algorithm.
- void aco_free_pheromone (double **p, int n)

  Free a pheromone matrix created by aco_create_pheromone().
- void construct_solution (ACO *aco, Flowshop *fs, Ant *ant)

  Construct a new ant solution based on pheromone and heuristic information.
- void update_pheromone (ACO *aco, Ant *ants, int num_ants, int n_jobs, int best_makespan)

  Update pheromone levels using evaporation and deposition from all ants.
- void local_search_insertion (Flowshop *fs, Ant *ant, method choose)

  Perform a local-search insertion improvement on an ant's solution.
- Solution ant_colony_optimize (Flowshop *fs, Solution initial, method choose, Config cfg)

  Run the Ant Colony Optimization loop and return the best solution found.

5.15.1 Detailed Description

Ant Colony Optimization implementation for flow shop scheduling.

This module implements pheromone initialization, probabilistic solution construction, randomized local search, pheromone updates, and the main ACO optimization loop for both standard and blocking objectives.

5.15.2 Function Documentation

5.15.2.1 aco_create_pheromone()

```
double ** aco_create_pheromone (
            int n,
            Solution neh)
```

Create and initialize the pheromone matrix for the Ant Colony algorithm.

Allocate and initialize the pheromone transition matrix.

The pheromone matrix is sized (n+1) x n to include a virtual "start" node at index n. The NEH solution is used to seed pheromone on the transitions it uses.

Parameters

	n	Number of jobs.
	neh	Initial NEH solution used to seed pheromone.

Returns

Newly allocated pheromone matrix (caller must free).

5.15.2.2 aco_free_pheromone()

```
void aco_free_pheromone (
            double ** p,
            int n)
```

Free a pheromone matrix created by aco_create_pheromone().

Parameters

	p	Pointer to the pheromone matrix.
	n	Number of jobs used for per-row cleanup.

5.15.2.3 ant_colony_optimize()

```
Solution ant_colony_optimize (
            Flowshop * fs,
            Solution initial,
            method choose,
            Config cfg)
```

Run the Ant Colony Optimization loop and return the best solution found.

Run Ant Colony Optimization starting from an initial solution.

Uses pheromone initialization from the provided initial solution, constructs new ant solutions each iteration, optionally improves them with local search, and updates pheromone based on the best tour found.

Parameters

	fs	Flowshop problem instance.
	initial	Initial solution used to seed pheromone and as the starting best.
	choose	Evaluation method (standard or blocking makespan).
	cfg	Configuration values controlling ACO behavior.

Returns

Best solution found by ACO. The caller is responsible for freeing the returned schedule.

5.15.2.4 construct_solution()

```
void construct_solution (
            ACO * aco,
            Flowshop * fs,
            Ant * ant)
```

Construct a new ant solution based on pheromone and heuristic information.

Build a single ant solution from pheromone and heuristic values.

This uses a job-to-job transition graph and selects the next job by roulette wheel selection using pheromone^alpha * heuristic^beta.

Parameters

	aco	Pointer to the ACO algorithm state.
	fs	Flowshop problem instance.
	ant	Ant object to fill with a constructed permutation.

5.15.2.5 evaluate_blocking()

```
int evaluate_blocking (
            Flowshop * fs,
            int * perm)
```

Evaluate a schedule using the blocking makespan calculation.

Evaluate a permutation with the blocking flow shop makespan model.

Parameters

	fs	Pointer to the Flowshop instance.
	perm	Permutation of job indices representing the schedule.

Returns

The makespan for the given schedule (blocking variant).

5.15.2.6 evaluate_standard()

```
int evaluate_standard (
            Flowshop * fs,
            int * perm)
```

Evaluate a schedule using the standard (non-blocking) makespan calculation.

Evaluate a permutation with the standard flow shop makespan model.

Parameters

	fs	Pointer to the Flowshop instance.
	perm	Permutation of job indices representing the schedule.

Returns

The makespan for the given schedule.

5.15.2.7 local_search_insertion()

```
void local_search_insertion (
            Flowshop * fs,
            Ant * ant,
            method choose)
```

Perform a local-search insertion improvement on an ant's solution.

Tries random insertion moves and accepts any move that yields a strictly better makespan, as computed by `choose`.

Parameters

fs	Flowshop problem instance.
ant	Ant whose permutation is being improved.
choose	Pointer to the evaluation method (standard or blocking).

5.15.2.8 update_pheromone()

```
void update_pheromone (
            ACO * aco,
            Ant * ants,
            int num_ants,
            int n_jobs,
            int best_makespan)
```

Update pheromone levels using evaporation and deposition from all ants.

Apply evaporation and deposition to the pheromone matrix.

Deposition is scaled by (best_makespan / ant_makespan) so better solutions deposit more pheromone.

Parameters

aco	Pointer to the ACO state containing pheromone and rho.
ants	Array of ants whose tours are used for deposition.
num_ants	Number of ants in the array.
n_jobs	Number of jobs defining pheromone matrix dimensions.
best_makespan	Best makespan found in this iteration for scaling.

5.16 src/blocking.c File Reference

Blocking flow shop makespan and NEH heuristic implementation.

```
#include "FLOWSHOP.h"
#include "blocking.h"
#include "mt19937ar.h"
```

Functions

- int flowshop_calculate_makespan_blocking (Flowshop *fs, int *schedule, int count)

  Compute the blocking flow shop makespan for a partial schedule.
- Solution flowshop_schedule_blocking (Flowshop *fs)

  Construct a schedule using the blocking NEH heuristic.

5.16.1 Detailed Description

Blocking flow shop makespan and NEH heuristic implementation.

This module evaluates schedules under blocking constraints and builds blocking-aware NEH solutions with randomized tie-breaking.

5.16.2 Function Documentation

5.16.2.1 flowshop_calculate_makespan_blocking()

```
int flowshop_calculate_makespan_blocking (
            Flowshop * fs,
            int * schedule,
            int count)
```

Compute the blocking flow shop makespan for a partial schedule.

Parameters

	fs	Flow shop instance being evaluated.
	schedule	Permutation of job indices.
	count	Number of leading jobs from `schedule` to evaluate.

Returns

Blocking makespan of the partial schedule.

5.16.2.2 flowshop_schedule_blocking()

```
Solution flowshop_schedule_blocking (
            Flowshop * fs)
```

Construct a schedule using the blocking NEH heuristic.

Build a blocking NEH schedule for the given flow shop instance.

Jobs are ranked by total processing time and inserted iteratively at the position that minimizes the blocking makespan, with random tie-breaking.

Parameters

	fs	Flow shop instance to schedule.

Returns

Solution produced by the blocking NEH heuristic.

5.17 src/config.c File Reference

Configuration file loader for Project 4.

```
#include <stdio.h>
#include <string.h>
#include "config.h"
```

Functions

- Config load_config (const char *filename)

  Load configuration values from a text file.

5.17.1 Detailed Description

Configuration file loader for Project 4.

This module parses a simple text configuration file containing numeric `key=value` pairs and populates a Config structure with either the parsed values or built-in defaults.

5.17.2 Function Documentation

5.17.2.1 load_config()

```
Config load_config (
            const char * filename)
```

Load configuration values from a text file.

Supported keys include `numfiles`, `ants`, `iterations`, `alpha`, `beta`, and `rho`. Missing files cause the function to return a Config structure populated with default values.

Parameters

	filename	Path to the configuration file to read.

Returns

Parsed configuration values, or defaults if the file cannot be opened.

5.18 src/fileutility.c File Reference

File-loading helpers for flow shop benchmark instances.

```
#include <stdio.h>
#include <stdlib.h>
#include "FLOWSHOP.h"
```

Functions

- Flowshop * loadVector (const char *filename)

  Load a flow shop instance from an input file.

5.18.1 Detailed Description

File-loading helpers for flow shop benchmark instances.

This module reads instance files from disk and converts the machine-row input format into per-job processing-time arrays stored in a Flowshop.

5.18.2 Function Documentation

5.18.2.1 loadVector()

```
Flowshop * loadVector (
            const char * filename)
```

Load a flow shop instance from an input file.

Load a flow shop instance from a text file.

The file format begins with the number of machines and jobs, followed by a matrix of processing times stored one machine row at a time.

Parameters

	filename	Path to the input file to read.

Returns

Newly allocated flow shop instance, or NULL if loading fails.

5.19 src/flowshop.c File Reference

Standard flow shop data structure and NEH heuristic implementation.

```
#include <stdlib.h>
#include "FLOWSHOP.h"
#include "stdio.h"
#include <string.h>
#include "mt19937ar.h"
```

Functions

- Flowshop * flowshop_create (int num_machines, int num_jobs)

  Allocate and initialize a flow shop instance.
- void flowshop_free (Flowshop *fs)

  Release all memory owned by a flow shop instance.
- void flowshop_add_job (Flowshop *fs, int job_id, int *times)

  Insert a job definition into the next free slot of a flow shop.
- int compare_neh (const void *a, const void *b)

  Compare two JobRank entries by descending total processing time.
- int flowshop_calculate_makespan_partial (Flowshop *fs, int *schedule, int n_jobs)

  Compute the standard flow shop makespan for a partial permutation.
- Solution flowshop_schedule (Flowshop *fs)

  Construct a schedule using the standard NEH heuristic.

5.19.1 Detailed Description

Standard flow shop data structure and NEH heuristic implementation.

This module manages flow shop instances, computes standard makespans, and constructs schedules using the NEH heuristic with randomized tie-breaking.

5.19.2 Function Documentation

5.19.2.1 compare_neh()

```
int compare_neh (
            const void * a,
            const void * b)
```

Compare two JobRank entries by descending total processing time.

Compare two JobRank values by descending total processing time.

Parameters

	a	Pointer to the first JobRank.
	b	Pointer to the second JobRank.

Returns

Negative, zero, or positive according to qsort comparator rules.

5.19.2.2 flowshop_add_job()

```
void flowshop_add_job (
            Flowshop * fs,
            int job_id,
            int * times)
```

Insert a job definition into the next free slot of a flow shop.

Add a job and its processing times to a flow shop instance.

Parameters

	fs	Flow shop instance receiving the job.
	job_id	External identifier assigned to the job.
	times	Processing-time array with one entry per machine.

5.19.2.3 flowshop_calculate_makespan_partial()

```
int flowshop_calculate_makespan_partial (
            Flowshop * fs,
            int * schedule,
            int n_jobs)
```

Compute the standard flow shop makespan for a partial permutation.

Compute the standard flow shop makespan for a partial schedule.

Parameters

fs	Flow shop instance being evaluated.	
schedule	Permutation of job indices.	
n_jobs	Number of leading jobs from `schedule` to evaluate.	

Returns

Makespan of the partial schedule.

5.19.2.4 flowshop_create()

```
Flowshop * flowshop_create (
            int num_machines,
            int num_jobs)
```

Allocate and initialize a flow shop instance.

Parameters

num_machines	Number of machines in the instance.	
num_jobs	Number of jobs in the instance.	

Returns

Newly allocated flow shop instance.

5.19.2.5 flowshop_free()

```
void flowshop_free (
            Flowshop * fs)
```

Release all memory owned by a flow shop instance.

Parameters

fs	Flow shop instance to free.	

5.19.2.6 flowshop_schedule()

```
Solution flowshop_schedule (
            Flowshop * fs)
```

Construct a schedule using the standard NEH heuristic.

Build a standard NEH schedule for the given flow shop instance.

Jobs are ranked by total processing time and inserted iteratively at the position that minimizes the partial makespan, with MT19937 tie-breaking.

Parameters

	fs	Flow shop instance to schedule.

Returns

Solution produced by the standard NEH heuristic.

5.20 src/main.c File Reference

Benchmark driver for NEH and Ant Colony Optimization flow shop solvers.

```
#include <stdio.h>
#include <stdlib.h>
#include "FLOWSHOP.h"
#include "mt19937ar.h"
#include "blocking.h"
#include "fileutility.h"
#include "timing.h"
#include "ACO.h"
#include "config.h"
```

Functions

- void write_schedule_csv (FILE *file, int instance, const char *algorithm, Solution sol, Flowshop *fs)

  Write a solution schedule to the schedules CSV output file.
- int main (int argc, char *argv[])

  Program entry point.

5.20.1 Detailed Description

Benchmark driver for NEH and Ant Colony Optimization flow shop solvers.

This program loads a sequence of flow shop instances, runs both the standard and blocking NEH heuristics, improves them with ACO, and writes benchmark results and schedules to CSV output files.

5.20.2 Function Documentation

5.20.2.1 main()

```
int main (
            int argc,
            char * argv[])
```

Program entry point.

Expects a single command-line argument specifying how many numbered input instances to process. The configuration values are loaded from `config.cfg`.

Parameters

	argc	Argument count.
	argv	Argument vector.

Returns

Exit status code (0 on success).

5.20.2.2 write_schedule_csv()

```
void write_schedule_csv (
            FILE * file,
            int instance,
            const char * algorithm,
            Solution sol,
            Flowshop * fs)
```

Write a solution schedule to the schedules CSV output file.

Each row contains the benchmark instance identifier, algorithm label, solution makespan, and the scheduled job IDs in order.

Parameters

	file	Output stream for the CSV file.
	instance	Instance number associated with the solution.
	algorithm	Label identifying the algorithm variant.
	sol	Solution containing the permutation and makespan.
	fs	Flow shop instance used to map permutation indices to job IDs.

5.21 src/mt19937ar.c File Reference

Mersenne Twister MT19937 random number generator.

```
#include "mt19937ar.h"
```

Macros

- #define **N** 624
- #define **M** 397
- #define **MATRIX_A** 0x9908b0dfUL
- #define **UPPER_MASK** 0x80000000UL
- #define **LOWER_MASK** 0x7fffffffUL

Functions

- void init_genrand (uint32_t s)

  Initializes the generator with a seed.
- uint32_t genrand_int32 (void)

  Generates a 32-bit unsigned random integer.
- double genrand_real2 (void)

  Generates a floating-point random number in [0, 1).

5.21.1 Detailed Description

Mersenne Twister MT19937 random number generator.

This file provides an implementation of the MT19937 pseudorandom number generator. It supports initialization with a seed and generation of 32-bit integers and double-precision floating-point values in the range [0,1).

Based on the original MT19937 reference implementation.

5.21.2 Function Documentation

5.21.2.1 genrand_int32()

```
uint32_t genrand_int32 (
            void )
```

Generates a 32-bit unsigned random integer.

If the generator state has been exhausted, the state is regenerated automatically.

Returns

Random 32-bit unsigned integer.

5.21.2.2 genrand_real2()

```
double genrand_real2 (
            void )
```

Generates a floating-point random number in [0, 1).

Generates a floating-point random number in the range [0, 1).

Uses a 32-bit integer random value scaled to the unit interval.

Returns

Random double in the range [0,1).

5.21.2.3 init_genrand()

```
void init_genrand (
            uint32_t s)
```

Initializes the generator with a seed.

Initializes the random number generator with a seed.

This function must be called before using the random number generator unless a default seed is acceptable.

Parameters

	s	Seed value.

5.22 src/timing.c File Reference

Cross-platform wall-clock timing utilities.

```
#include "timing.h"
#include <time.h>
```

Functions

- double now_ms (void)

  Returns the current wall-clock time in milliseconds (POSIX).

5.22.1 Detailed Description

Cross-platform wall-clock timing utilities.

This module provides a portable function for obtaining the current wall-clock time in milliseconds. Platform-specific implementations are selected at compile time.

5.22.2 Function Documentation

5.22.2.1 now_ms()

```
double now_ms (
            void )
```

Returns the current wall-clock time in milliseconds (POSIX).

Returns the current wall-clock time in milliseconds.

Uses CLOCK_MONOTONIC when available, otherwise falls back to clock().

Returns

Current time in milliseconds.

Index